Task & To-Do Management System

Mid-Semester Project Report

Software Engineering

Submitted by

Name	Registration No.
Rounak Das	20243585
Priyansh Agarwal	20243537
Shashank Karn	20243546
Shivam Verma	20243263

Date: 10-19-2026

1. Problem Statement and Motivation

Students and small teams usually have a lot of tasks going on at the same time, like assignments, lab work, project work and personal to-dos. Most of the time these tasks are kept in notebooks, phone notes or WhatsApp messages. Because of this, tasks get forgotten, deadlines are missed and it is hard to see what is done and what is still left.

There are big tools like Jira and Trello, but they have a lot of features that a student does not need, and some of them need a paid plan for team features. We wanted to build a simple web app where a user can make an account, add their tasks and see them on a Kanban board with three columns: **To Do**, **In Progress** and **Done**.

This project also lets our team practise the software engineering process that we learned in class, like requirements, design, Agile sprints and testing, on a real working system.

2. Project Objectives

1. Build a web application where users can sign up and log in securely.
2. Let users add, view, edit and delete their tasks with a title, description, priority, due date and status.
3. Show the tasks on a Kanban board with To Do, In Progress and Done columns.
4. Keep each user's tasks private so that one user cannot see or change another user's tasks.
5. Follow an Agile approach with short sprints and a product backlog.
6. Write automated tests for the backend and do manual testing of the UI.

3. Scope of the Project

3.1 Features Included

- User sign up, log in and log out
- Add, view, edit and delete tasks
- Task fields: title, description, priority (low / medium / high), due date and status
- Kanban board with three columns (To Do, In Progress, Done)
- Changing the status of a task from the card
- Due date shown and entered in MM-DD-YYYY format
- Search and filter tasks (planned for the second half)
- Sprints with a progress chart (planned for the second half)
- Overdue highlighting and dark mode (planned for the second half)

3.2 Features Excluded

- Mobile app (only a web app is made)

- Sharing tasks between users or team workspaces
- Email or push notifications and reminders
- File attachments on tasks
- Password reset through email
- Online deployment (the project runs locally)

3.3 Assumptions and Constraints

- Users have a modern web browser (Chrome, Firefox, Edge or Safari).
- Node.js 20 or newer is installed on the machine that runs the project.
- The app runs on a local machine, so only a small number of users is expected.
- The project has to be done in one semester by a team of four students.
- Only free and open-source tools are used.

4. Stakeholder Identification

Stakeholder	Role / Interest
Students / end users	Use the app to keep track of their tasks and deadlines.
Project team (4 members)	Design, build, test and document the system.
Course instructor	Evaluates the project and gives feedback.
Teaching assistants	Help with reviews and checking progress.

5. Requirements Specification

5.1 Functional Requirements

ID	Requirement
FR1	The system shall allow a new user to sign up with name, email and password.
FR2	The system shall not allow two accounts with the same email.
FR3	The system shall allow a registered user to log in with email and password.
FR4	The system shall allow a logged in user to log out.
FR5	The system shall allow a user to add a task with title, description, priority, due date and status.
FR6	The system shall show the user’s tasks on a Kanban board with To Do, In Progress and Done columns.
FR7	The system shall allow a user to edit any of their tasks.
FR8	The system shall allow a user to change the status of a task to move it to another column.
FR9	The system shall allow a user to delete a task after confirming.
FR10	The system shall only show a user their own tasks.
FR11	The system shall show and accept due dates in MM-DD-YYYY format.
FR12	The system shall allow the user to search and filter tasks.

ID	Requirement
FR13	The system shall allow the user to group tasks into sprints and see a progress chart.
FR14	The system shall highlight overdue tasks.
FR15	The system shall support drag-and-drop between columns.
FR16	The system shall have a dark mode.

5.2 Non-Functional Requirements

ID	Type	Requirement
NFR1	Security	Passwords shall be stored as bcrypt hashes and never as plain text.
NFR2	Security	All task requests shall need a valid JWT token.
NFR3	Performance	Pages and API calls should respond in under 1 second on a normal laptop.
NFR4	Usability	A new user should be able to add their first task without any help.
NFR5	Reliability	Wrong input (empty title, bad date, wrong priority) shall show an error and not crash the app.
NFR6	Portability	The app shall run on Windows, macOS and Linux with Node.js.
NFR7	Maintainability	Code shall be split into frontend and backend folders with small modules.

5.3 Prioritisation of Requirements

We used the **MoSCoW** method to prioritise the requirements. This list also works as our product backlog.

Priority	Requirements	Status
Must have	FR1, FR2, FR3, FR4, FR5, FR6, FR7, FR8, FR9, FR10, NFR1, NFR2	Done
Should have	FR11, FR12, NFR5	FR11 and NFR5 done, FR12 planned
Could have	FR13, FR14, FR15, FR16	Planned
Won’t have (this semester)	The excluded features in Section 3.2	Not planned

6. Requirements Model

6.1 Use-Case Diagram

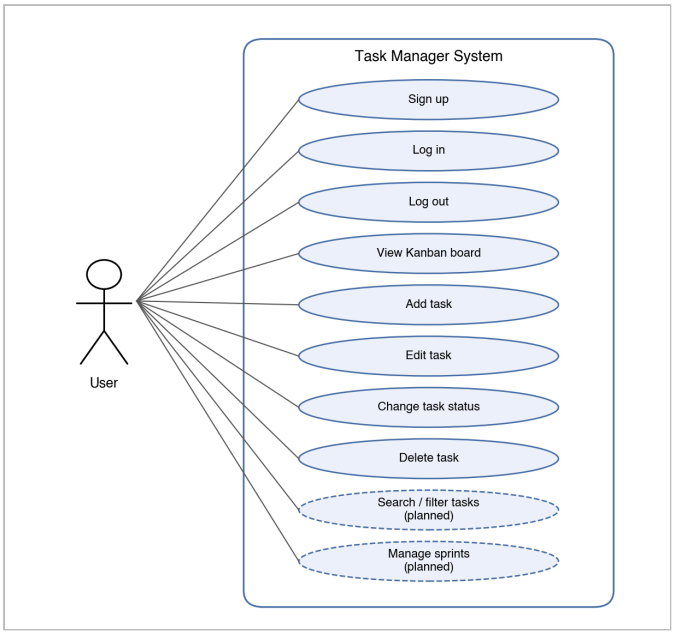


Figure 1: Use-case diagram

6.2 Use-Case Descriptions

ID	Use case	Actor	Precondition	Main flow	Alternate flow
UC1	Sign up	User	User does not have an account	User enters name, email and password and clicks Sign Up. Account is made and the board opens.	Email already used or password under 6 characters: error message is shown.
UC2	Log in	User	User has an account	User enters email and password and clicks Log In. Board is shown.	Wrong email or password: “Wrong email or password” is shown.
UC3	Add task	User	User is logged in	User clicks “+ Add Task”, fills the form and clicks Save. Task shows in the right column.	Empty title or bad date: error is shown in the form.
UC4	Edit task	User	Task exists	User clicks Edit, changes fields and clicks Save. Card is updated.	Same validation as UC3.
UC5	Change status	User	Task exists	User picks a new status from the dropdown on the card. Card moves to that column.	Server error: message shown on the board.
UC6	Delete task	User	Task exists	User clicks Delete and confirms. Card is removed.	User cancels: nothing happens.

7. System Design

7.1 Architecture

The system uses a simple **client-server (three-tier) architecture**. The React frontend runs in the browser and talks to the Express backend through a REST API using JSON. The backend checks the JWT token, runs the logic and reads or writes data in a SQLite database file.

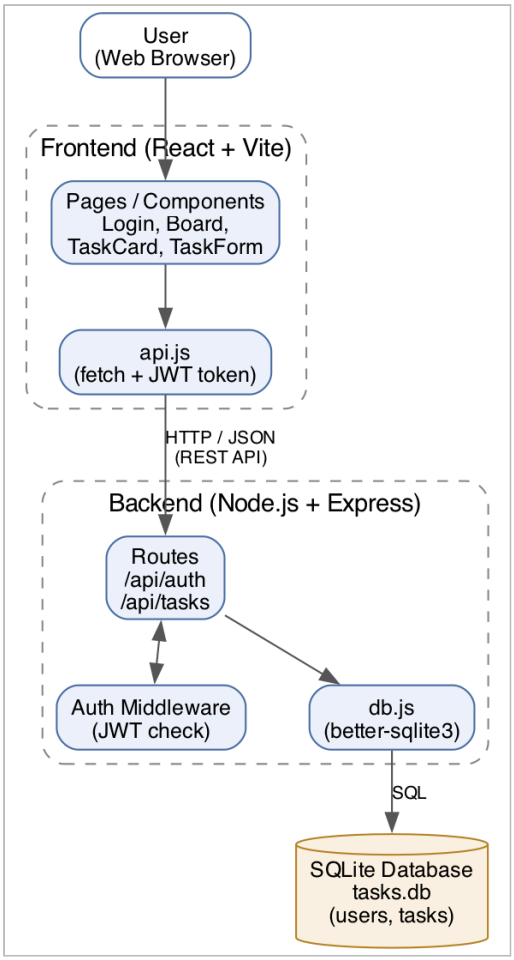


Figure 2: System architecture

7.2 Module Description

Module	File(s)	Description
Login page	client/src/components/Login.jsx	Log in and sign up form. Saves the token after login.
Board	client/src/components/Board.jsx	Loads tasks and shows them in three columns. Handles add, edit, status change and delete.
Task card	client/src/components/TaskCard.jsx	Shows one task with priority, due date, status dropdown, Edit and Delete buttons.
Task form	client/src/components/TaskForm.jsx	Popup form to add or edit a task. Checks the MM-DD-YYYY date format.

Module	File(s)	Description
API helper	client/src/api.js	All fetch calls to the backend. Adds the JWT token to every request.
Auth routes	server/src/routes/auth.js	/register, /login and /me. Hashes passwords with bcrypt and makes JWT tokens.
Task routes	server/src/routes/tasks.js	GET, POST, PUT and DELETE for tasks. Validates the data.
Auth middleware	server/src/middleware/auth.js	Checks the JWT token and puts the user id on the request.
Database	server/src/db.js	Opens the SQLite file and creates the tables if needed.
Seed script	server/scripts/initDb.js	Adds a demo user and some sample tasks.

7.3 REST API

Method	URL	Login needed	Description
POST	/api/auth/register	No	Make a new account, returns token and user
POST	/api/auth/login	No	Log in, returns token and user
GET	/api/auth/me	Yes	Get the logged in user
GET	/api/tasks	Yes	Get all tasks of the user
POST	/api/tasks	Yes	Add a task
PUT	/api/tasks/:id	Yes	Update a task (any field)
DELETE	/api/tasks/:id	Yes	Delete a task

7.4 Database Design

The database has two tables. One user can have many tasks, and each task belongs to exactly one user through the `user_id` foreign key. Due dates are stored as YYYY-MM-DD so that they sort correctly, and the frontend changes them to MM-DD-YYYY for the user.

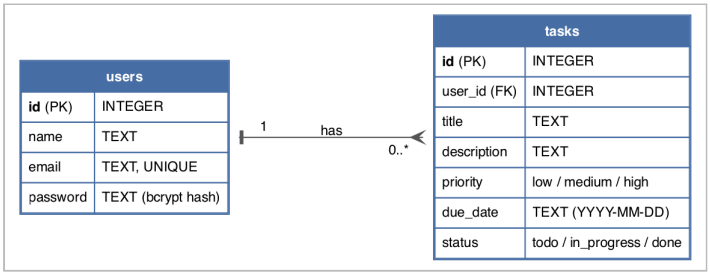


Figure 3: ER diagram

7.5 Sequence Diagram: Log In

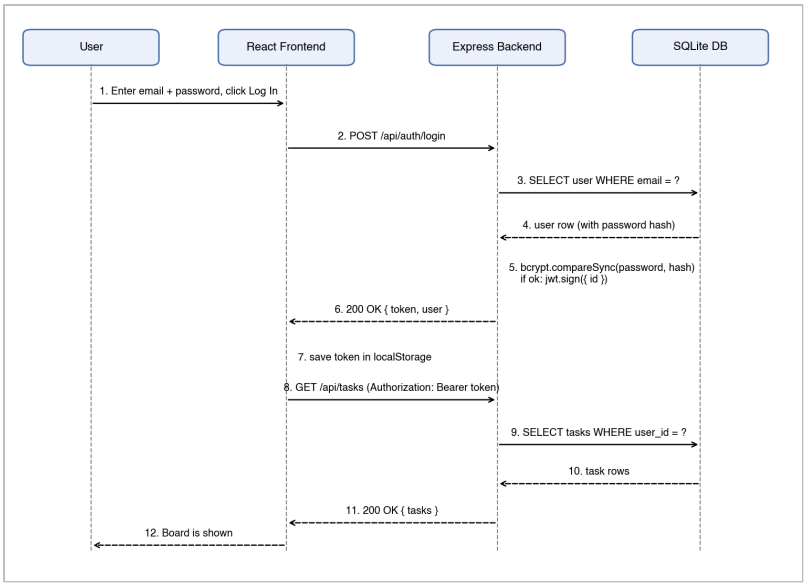


Figure 4: Sequence diagram for logging in and loading the board

7.6 Activity Diagram: Add Task

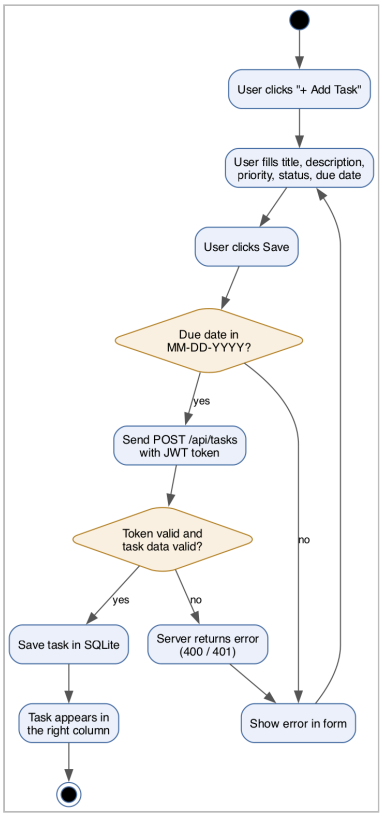


Figure 5: Activity diagram for adding a task

8. Technology Stack and Justification

Part	Technology	Why we chose it
Frontend	React 18 with Vite	Most of the team already knew some React. Components make the board, card and form easy to split up. Vite starts fast and has a simple proxy for the API.
Backend	Node.js + Express	Same language (JavaScript) as the frontend, so everyone can read all the code. Express is small and easy to learn.
Database	SQLite (better-sqlite3)	No database server to install, the whole database is one file. Good enough for a small number of users.
Login	JWT + bcrypt	JWT tokens are simple to use with a REST API. bcrypt is a standard way to hash passwords.
Testing	Jest + Supertest	Jest is the most common JavaScript test tool. Supertest lets us call the API in tests without starting the server.
Version control	Git + GitHub	Each member works on branches and we can see who did what.

9. Implementation Progress

9.1 Features Completed

- Sign up, log in and log out with JWT tokens and bcrypt password hashing
- Add, edit and delete tasks
- Kanban board with To Do, In Progress and Done columns and a task count for each column
- Changing task status from a dropdown on the card
- Due dates in MM-DD-YYYY format with validation on both frontend and backend
- Each user only sees their own tasks
- Sample data script with a demo user
- 15 automated backend tests

This means our **working end-to-end feature** is task management: the user logs in on the React page, the request goes to the Express API, the task is saved in SQLite and then shown on the board.

9.2 Features Under Development

- Search and filter tasks by title, priority and status
- Drag-and-drop between columns
- Sprints with a progress chart of tasks left
- Highlighting overdue tasks
- Dark mode

9.3 Screenshots of the Working System

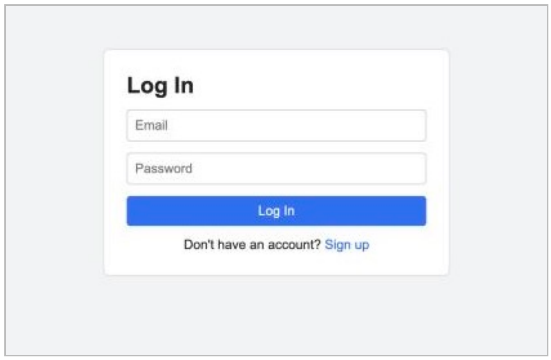


Figure 6: Log in page

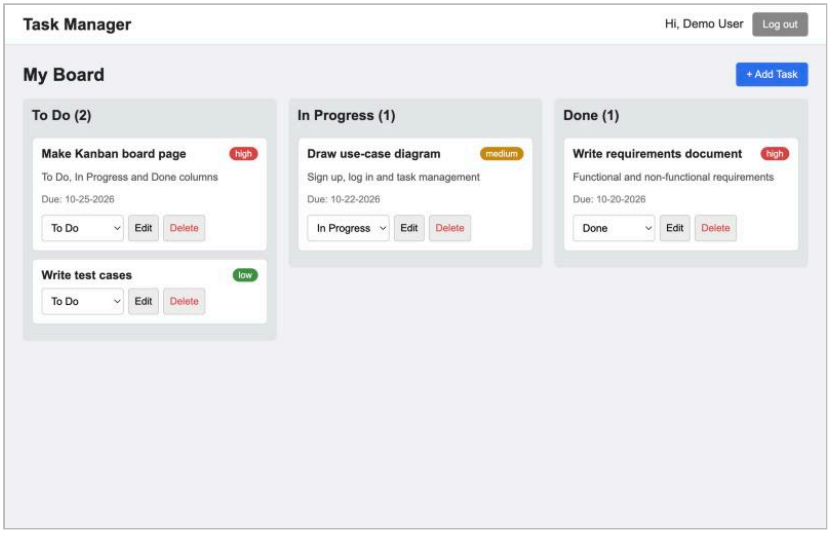


Figure 7: Kanban board after logging in with the demo user

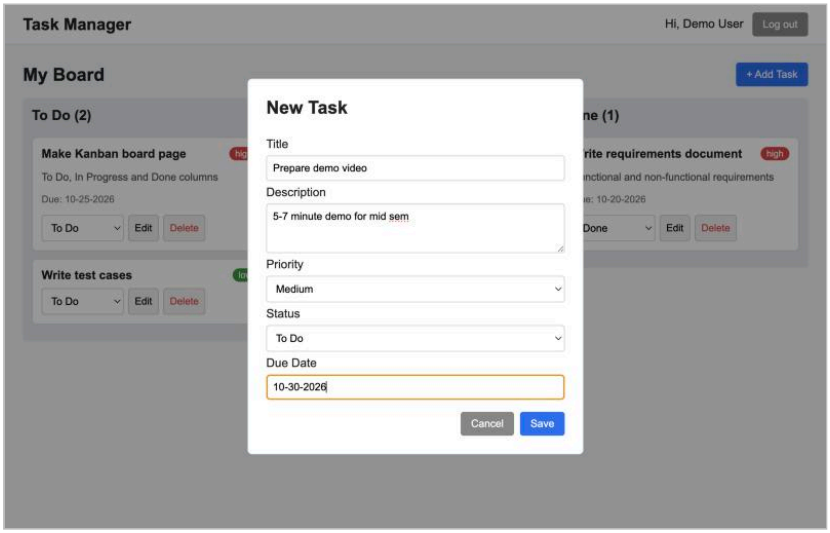


Figure 8: Add task form

9.4 Important Implementation Decisions

1. **JWT instead of sessions.** The server does not need to remember who is logged in. The token is saved in the browser's localStorage and sent with each request.
2. **Every task query checks `user_id`.** All SELECT, UPDATE and DELETE queries use `WHERE user_id = ?`, so a user can never change another user's task even if they guess the task id.
3. **Dates stored as YYYY-MM-DD, shown as MM-DD-YYYY.** The database format sorts correctly and the user sees the format they asked for. We used a text box instead of the browser date picker because the date picker shows the date in the computer's own format (for example DD/MM/YYYY), not MM-DD-YYYY.
4. **Validation on both sides.** The form checks the date format before sending, and the server checks the title, priority, status and date again, so bad data cannot get in even if someone calls the API directly.

10. Testing Performed

We did two kinds of testing:

- **Automated tests** for the backend API using Jest and Supertest. Each test file uses a fresh in-memory SQLite database.
- **Manual testing** of the UI in Google Chrome using the demo user.

10.1 Test Cases

ID	Test case	Input	Expected result	Actual result	Status
T1	Sign up	name, email, password "secret123"	201, token and user returned	201, token and user returned	Pass
T2	Duplicate email	Same email again	400 error	400 error	Pass
T3	Short password	Password "123"	400 error	400 error	Pass
T4	Log in	Correct email and password	200, token returned	200, token returned	Pass
T5	Wrong password	Password "wrongpass"	401 error	401 error	Pass
T6	/me without token	No Authorization header	401 error	401 error	Pass
T7	Tasks without login	GET /api/tasks, no token	401 error	401 error	Pass
T8	Add task	title "Write SRS", priority high, due date	201, status "todo"	201, status "todo"	Pass
T9	Add task without title	description only	400 error	400 error	Pass

ID	Test case	Input	Expected result	Actual result	Status
T10	Wrong priority	priority “urgent”	400 error	400 error	Pass
T11	Wrong due date	due_date “2026-13-45”	400 error	400 error	Pass
T12	List tasks	GET /api/tasks	1 task returned	1 task returned	Pass
T13	Change status	PUT status “done”	status “done”, title same	status “done”, title same	Pass
T14	Other user’s task	Second user lists and deletes	List empty, delete gives 404	List empty, delete gives 404	Pass
T15	Delete task	DELETE /api/tasks/:id	200, task gone from list	200, task gone from list	Pass
M1	Log in from UI	demo@example.com / password123	Board with sample tasks	Board with sample tasks	Pass
M2	Wrong password in UI	demo@example.com / wrongpass	“Wrong email or password”	“Wrong email or password”	Pass
M3	Sign up from UI	New name, email, password	Empty board opens	Empty board opens	Pass
M4	Add task from UI	Title, description, date 10-30-2026	Card in To Do with “Due: 10-30-2026”	Card in To Do with “Due: 10-30-2026”	Pass
M5	Wrong date format	Date “2026/10/30”	“Due date must be in MM-DD-YYYY format”	Message shown, task not saved	Pass
M6	Impossible date	Date “13-45-2026”	“Due date is not valid”	Message shown, task not saved	Pass
M7	Edit task	Click Edit, change title	Form filled with old values, card updated	Form filled (date as 10-30-2026), card updated	Pass
M8	Change status from card	Pick “In Progress”	Card moves to In Progress	Card moves to In Progress	Pass
M9	Log out	Click Log out	Log in page shown	Log in page shown	Pass

T1 to T15 are automated tests and M1 to M9 are manual tests. **All 24 test cases passed.**

10.2 Evidence

Output of `npm test` in the server folder:

```
PASS tests/auth.test.js
  Auth
    ✓ user can sign up
    ✓ cannot sign up twice with the same email
    ✓ cannot sign up with a short password
    ✓ user can log in
    ✓ cannot log in with wrong password
    ✓ cannot get user without token

PASS tests/tasks.test.js
  Tasks
    ✓ cannot get tasks without logging in
    ✓ user can add a task
    ✓ cannot add a task without a title
    ✓ cannot add a task with a wrong priority
    ✓ cannot add a task with a wrong due date
    ✓ user can see their tasks
    ✓ user can move a task to done
    ✓ other user cannot see or delete the task
    ✓ user can delete a task

Test Suites: 2 passed, 2 total
Tests:       15 passed, 15 total
```

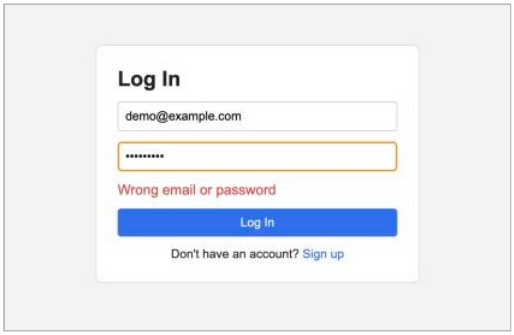


Figure 9: Test M2, wrong password message

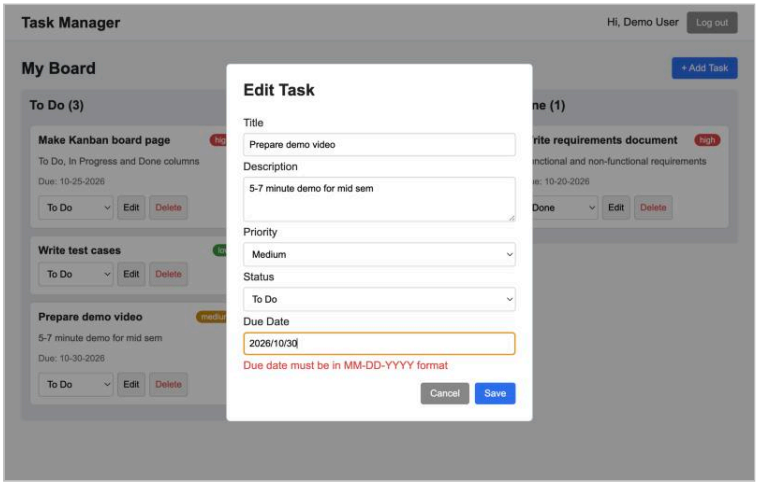


Figure 10: Test M5, wrong date format message

11. Project-Management Status

11.1 Original Project Plan

We follow Agile with two sprints, as planned at the start of the project.

Phase	Work	Planned
Planning	Choose topic, problem statement, stakeholders, requirements, backlog	Before mid-sem
Design	Architecture, ER diagram, use cases, UI wireframes	Before mid-sem
Sprint 1	Login and sign up, task add / edit / delete, Kanban board, backend tests	Before mid-sem
Sprint 2	Search and filter, drag-and-drop, sprints and progress chart, overdue highlighting, dark mode	After mid-sem
Final	Final testing, final report, final demo	End of semester

11.2 Current Progress

Work	Status
Problem statement, scope and requirements	Done
Design diagrams (architecture, ER, use case, sequence, activity)	Done
Sprint 1 features	Done
Backend automated tests	Done (15 tests passing)
Manual UI testing	Done (9 test cases)
Sprint 2 features	Not started

Overall we think the project is about **50% done**.

11.3 Delayed or Pending Tasks

- Drag-and-drop was moved to Sprint 2. For now the status is changed with a dropdown, which was quicker to build and test.
- Frontend tests (React components) are not written yet.

11.4 Risks and Mitigation

Risk	Impact	Mitigation
Team members busy with other exams and assignments	Sprint 2 may be late	Keep Sprint 2 tasks small and do the must-have ones first
Drag-and-drop library is hard to learn	Feature might not finish	Status dropdown already works as a backup

Risk	Impact	Mitigation
Different Node.js versions on members' laptops	Project might not run for everyone	README says Node.js 20 or newer, everyone uses the same version
Merge conflicts in Git	Lost time	Small branches, pull often, frontend and backend owned by different members
Losing data in the local database	Demo might break	`npm run db:init` makes the sample data again in a few seconds

12. Individual Contribution

Member	Role	Work
Rounak Das	Backend and database	Express server, login and sign up with JWT and bcrypt, task routes, SQLite database, sample data, backend tests, first version of the frontend, report draft, database design
Priyansh Agarwal	Testing and diagrams	Test case table, test evidence, UML diagrams (use case, sequence, activity), architecture diagram, use case descriptions
Shashank Karn	Frontend and UI	UI fixes, wireframes, demo recording and editing, demo video link
Shivam Verma	Report and project management	SRS, final report, project plan, meeting log, contribution record, final ZIP and Moodle upload

13. Challenges Faced and Solutions

Challenge	What we did
The browser date picker showed dates in the computer's format (DD/MM/YYYY), but we wanted MM-DD-YYYY.	Used a normal text box with an MM-DD-YYYY placeholder and converted the date to YYYY-MM-DD before saving.
The SQLite library (better-sqlite3) did not install on older Node.js versions.	Agreed that everyone uses Node.js 20 or newer and wrote it in the README.
Making sure one user cannot see another user's tasks.	Added `user_id` to every task query and wrote a test for it (T14).
Running `npm install` in the wrong folder (repo root) gave errors.	Wrote clear steps in the README to run commands inside `server` and `client`.

14. Work Planned for the Remaining Semester

1. Search and filter tasks by title, priority and status.
2. Drag-and-drop to move cards between columns.
3. Sprints: create a sprint, add tasks to it and show a chart of tasks left.
4. Highlight tasks that are past their due date.
5. Dark mode.
6. Frontend tests and more backend tests.

7. Final report, final demo video and project presentation.

15. References

1. I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
2. K. Beck et al., “Manifesto for Agile Software Development,” 2001. https://agilemanifesto.org
3. React Documentation. https://react.dev
4. Vite Documentation. https://vitejs.dev
5. Express Documentation. https://expressjs.com
6. SQLite Documentation. https://www.sqlite.org/docs.html
7. better-sqlite3 Documentation. https://github.com/WiseLibs/better-sqlite3
8. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
9. Jest Documentation. https://jestjs.io
10. Supertest. https://github.com/ladjs/supertest

Appendix A: AI Tool Usage Disclosure

Item	Details
Tool	Claude (Anthropic), used through Claude Code
Purpose	Understanding the project requirements, technical guidance on the tech stack, generating parts of the code, diagrams and a first draft of this report
Nature of generated content	Starter code for the backend and frontend, test cases, diagram drawings and report text
Verification / modification by students	The team read and ran all the code, ran the automated tests, tested the app manually in the browser, and checked and edited the report

Appendix B: How to Run

```
cd server
npm install
cp .env.example .env
npm run db:init
npm run dev
```

In a second terminal:

```
cd client
npm install
npm run dev
```

Open http://localhost:5173 and log in with demo@example.com / password123.